CONTEXT Language Interpreter - Development Report & Interview Q&A

The build process, the questions it invites, terms answered, and sample code

Nirmalya Mandal - SAP Labs Interview Prep

Reconstructed from the development history

Contents

How to use this report

CONTEXT is small but punches above its weight in interviews: it proves you understand **how a language actually runs** - lexing, parsing, an AST, and evaluation. That is exactly the "fundamentals" signal SAP looks for, and it ties straight into compiler-design questions. Structure: **Part 1** build story, **Part 2** Q&A, **Part 3** bait-terms dictionary, **Part 4** sample code.

What it is in one breath: a small programming-language interpreter I built in Python from scratch - it reads code, breaks it into tokens, builds a syntax tree, and executes it, supporting arithmetic, variables, and if/else logic.

Why it's a strong hook: most students have never built one. Dropping "I wrote my own interpreter" invites great fundamentals questions you can own.

Part 1 - The development story

Phase 1 - The lexer (tokeniser)

- Built a **lexer** that reads the raw source text character by character and produces **tokens** - the smallest meaningful units (numbers, operators, parentheses).
- Handled **real numbers** (not just integers), so `3.5 + 2` tokenises correctly.

Phase 2 - Expression evaluation

- Implemented **expression evaluation** for arithmetic, respecting **operator precedence** (multiplication before addition).

Phase 3 - Variables

- Added **variable bindings** - assigning values to names and looking them up - backed by an **environment** (a name-to-value map).

Phase 4 - The AST

- Introduced an **Abstract Syntax Tree**: instead of evaluating text directly, the parser builds a tree of nodes (a number node, a binary-operation node), and an evaluator **walks** that tree.

Phase 5 - Booleans and control flow

- Added a **BooleanNode** and an **IfNode**, giving the language **conditional logic** (if/else), evaluated by walking the tree.
- Wrote an example script exercising the language.

Part 2 - Questions and detailed answers

"Walk me through how your interpreter runs a line of code."

Four stages. First the **lexer** turns the text into tokens - `3 + 4` becomes NUMBER, PLUS, NUMBER. Then the **parser** builds an **AST** from those tokens, respecting precedence, so it knows the structure and what to evaluate first. Then the **evaluator** walks that tree and computes the result, looking up any variables in the environment. It is the same pipeline real languages use, just small.

"What is a lexer? Token, lexeme, pattern?"

The lexer (or tokeniser) is the first stage; it scans the raw characters and groups them into tokens. A **token** is the category, like NUMBER or PLUS; the **lexeme** is the actual text matched, like `42` or `+`; the **pattern** is the rule that defines a token (a number is one or more digits, optionally with a decimal point).

"What is an AST, and how is it different from a parse tree?"

An Abstract Syntax Tree is a tree that captures the meaningful structure of the code. `3 + 4 * 2` becomes a tree with `+` at the root, `3` on the left, and a `*` subtree on the right - so precedence is baked into the shape. A parse tree shows every grammar rule applied and is more verbose; the AST keeps only what matters for evaluation.

"How do you handle operator precedence?"

Through the grammar and how the parser is structured. I parse expressions in layers: an expression is a sum of terms, a term is a product of factors, and a factor is a number or a parenthesised expression. Because factors and terms are parsed before sums, multiplication naturally binds tighter than addition. That is **recursive-descent** parsing.

"What is recursive-descent parsing?"

A top-down parsing style where each grammar rule is a function that calls the functions for the rules beneath it. `parse_expr` calls `parse_term`, which calls `parse_factor`. The recursion mirrors the grammar, and the call order enforces precedence.

"What is tree-walking evaluation?"

The evaluator recursively visits each AST node and computes its value. A number node returns its number; a binary-operation node evaluates its left and right children then applies the operator; an if node evaluates its condition and then the right branch. Walking the tree *is* running the program.

"Compiler versus interpreter - which is this?"

A compiler translates the whole program to machine code ahead of time; an interpreter reads and executes it directly. Mine is an **interpreter** - it walks the AST and executes on the spot, like Python does.

"How do variables work?"

An **environment** - a dictionary from names to values. Assignment stores a value under a name; using a variable looks it up. It is a simple symbol table.

Part 3 - The bait-terms dictionary

Lexer / tokeniser

First stage: text into tokens. *Say it:* "The lexer turns raw text into tokens like NUMBER and PLUS."

Token / lexeme / pattern

Category / matched text / rule. *Say it:* "Token is the type, lexeme is the actual text, pattern is the rule."

Parser / recursive descent

Builds the tree top-down, one function per rule. *Say it:* "A recursive-descent parser - each grammar rule is a function calling the ones below it."

AST vs parse tree

Meaningful structure vs full rule trace. *Say it:* "The AST keeps the meaningful structure with precedence baked into its shape."

Operator precedence

Binding order (times before plus). *Say it:* "Precedence falls out of parsing factors and terms before sums."

Tree-walking evaluator

Recursively evaluating each node. *Say it:* "The evaluator walks the tree - a number returns itself, an op node combines its children."

Environment / symbol table

Name-to-value map for variables. *Say it:* "Variables live in an environment - a name-to-value dictionary."

Compiler vs interpreter

Translate-ahead vs execute-directly. *Say it:* "Mine is an interpreter - it walks the AST and runs it directly."

Grammar

The rules defining valid programs. *Say it:* "The grammar defines expressions as sums of terms of factors."

Part 4 - Sample code snippets (pen and paper)

Python, kept tiny. Narrate the logic as you write.

The lexer (tokeniser)

```python
def tokenize(src):
    tokens, i = [], 0
    while i < len(src):
        c = src[i]
        if c.isspace():
            i += 1
        elif c.isdigit():
            num = ""
            while i < len(src) and (src[i].isdigit() or src[i] == "."):
                num += src[i]; i += 1
            tokens.append(("NUMBER", float(num)))
        elif c in "+-*/()":
            tokens.append((c, c)); i += 1
        else:
            raise SyntaxError(f"bad char {c}")
    return tokens
```

Say aloud: "It scans characters, gathering digits into a NUMBER token and mapping operators to their own tokens."

Recursive-descent parser (precedence via layers)

```python
# expr := term (('+'|'-') term)*
# term := factor (('*'|'/') factor)*
# factor := NUMBER | '(' expr ')'
def parse_expr(t):
    node = parse_term(t)
    while t.peek() in ("+", "-"):
        op = t.next()
        node = ("binop", op, node, parse_term(t))
    return node
```

Say aloud: "Because term is parsed before the plus loop, multiplication binds tighter - precedence for free."

Tree-walking evaluator

```python
def eval_node(node, env):
    kind = node[0]
```

```
    if kind == "num":
        return node[1]
    if kind == "var":
        return env[node[1]]
    if kind == "binop":
        _, op, a, b = node
        x, y = eval_node(a, env), eval_node(b, env)
        return {"+": x + y, "-": x - y,
                "*": x * y, "/": x / y}[op]
```

Say aloud: "It recurses: a number returns itself, a variable is looked up, an op evaluates both sides then applies the operator."

Evaluating an if node

```
def eval_if(node, env):
    _, cond, then_branch, else_branch = node
    if eval_node(cond, env):        # condition is truthy?
        return eval_node(then_branch, env)
    return eval_node(else_branch, env)
```

Say aloud: "The if node evaluates its condition, then walks the taken branch - control flow is just choosing which subtree to evaluate."

The hooks to drop on purpose

1. “I wrote my own language interpreter from scratch”
2. Lexer -> parser -> AST -> evaluator (the four stages)
3. Recursive-descent parsing and how it handles precedence
4. Tree-walking evaluation
5. Compiler vs interpreter

Every one has a full answer above - and this project is your cleanest bridge into any compiler-design question.